Informe Técnico — Agente Auditor Comunitario (Evaluación N°2)

Curso: ISY0101 Ingeniería de Soluciones con IA — Duoc UC, 2026-2 Estudiante: [COMPLETAR NOMBRE] Repositorio: https://github.com/SergioRafael0/agente-auditor-comunitario

1. Introducción y requerimiento organizacional

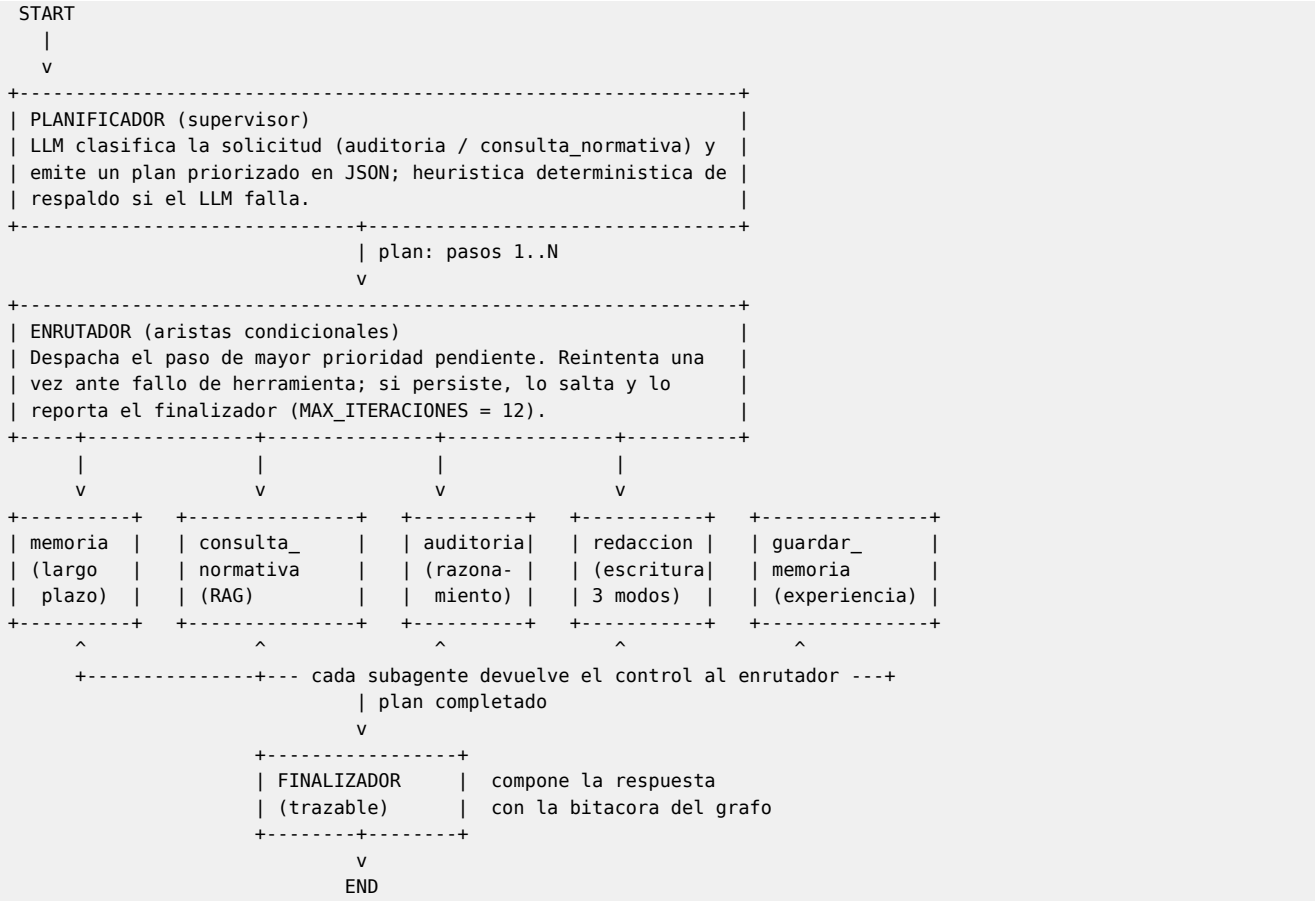
La plataforma simulada "Proveedores & Comunidades" permite que administradores de edificios publiquen necesidades de mantención para que proveedores les respondan. El riesgo organizacional es que dichas publicaciones no cumplan la normativa vigente —Ley 21.442 de Copropiedad Inmobiliaria (Ministerio de Vivienda y Urbanismo, 2021) y los Reglamentos de Instalaciones de Consumo de la SEC (Superintendencia de Electricidad y Combustibles, s. f.)—, exponiendo a la comunidad a sanciones y obras mal licitadas.

En la Evaluación N°1 se construyó un pipeline RAG + MCP server que audita publicaciones con una única función expuesta. La Evaluación N°2 lo evoluciona a un agente funcional: integra el framework LangGraph (LangChain AI, 2025) para orquestar herramientas de consulta, escritura y razonamiento que el sistema ejecuta con autonomía, incorpora memoria de corto y largo plazo, un esquema de planificación priorizada y decisiones adaptativas ante condiciones cambiantes, todo documentado y respaldado por pruebas.

Criterio de reuso: se conservaron sin cambios los componentes ya validados de la Evaluación 1 (indexación vectorial, retrieval semántico, cliente Groq y prompt v2 del auditor con few-shots), lo que redujo el riesgo del nuevo desarrollo a la capa de orquestación y memoria.

2. Diseño e implementación del agente (IE1, IE2)

La orquestación se implementó como un grafo dirigido en LangGraph (src/grafo/): un nodo planificador (supervisor) clasifica la solicitud y emite un plan priorizado, un enrutador recorre el plan y despacha subagentes especializados mediante aristas condicionales, y un finalizador compone la respuesta trazable. Cada subagente encapsula una única herramienta; el estado compartido y la bitácora de decisiones viajan por el grafo.



Herramientas del agente (IE1)

Tipo (rúbrica)	Herramienta	Módulo	Qué hace
Consulta	`buscar_normativa()`	`src/herramientas/consulta.py`	RAG sobre la normativa con

			citas exactas
Razonamiento			
	`auditar()`		
		`src/herramientas/razonamiento.py`	
			Veredicto `cumple/alerta/no_cumple` con fundamentación legal
Escritura			
	`redactar()` (3 modos)		
		`src/herramientas/redaccion.py`	
			Informe de auditoría, revisión de publicación o respuesta normativa

El agente las ejecuta con autonomía: el planificador decide el orden, el enrutador las invoca sin intervención humana y el modo de escritura se selecciona según el estado del caso. Extracto real de una ejecución (data/eval/ev2/evidencia_auditoria_normal.json, corrida contra la API de Groq):

```
[supervisor_planificador] Tipo de tarea 'auditoria'. Plan de 4 pasos priorizados.
[enrutador] Despacho paso 'memoria' (prioridad 1).
[subagente_memoria] Se recuperaron 3 experiencias previas.
[enrutador] Despacho paso 'auditoria' (prioridad 2).
[subagente_auditor] Veredicto del auditor: alerta (2 alertas).
[enrutador] Despacho paso 'redaccion' (prioridad 3).
[subagente_redactor] Informe de auditoria redactado.
[enrutador] Despacho paso 'guardar_memoria' (prioridad 4).
[subagente_guardar_memoria] Experiencia guardada en memoria de largo plazo.
[enrutador] Plan completado: finalizo y redacto resumen.
[finalizador] Respuesta final compuesta.
```

Elección del framework (IE2)

Se seleccionó LangGraph (ecosistema LangChain) por su modelo explícito de estado, sus aristas condicionales y su checkpointer nativo de memoria; alternativas descartadas: Haystack (pipelines rígidos, poco adecuados para decisiones adaptativas) y Autogen (conversaciones multi-LLM más costosas en tokens y latencia para este dominio). Escalabilidad: agregar un subagente nuevo equivale a registrar una herramienta en el diccionario NODOS_PASO, sin tocar el enrutador ni el estado. Compatibilidad: el MCP server de la Evaluación 1 se conserva como interfaz externa hacia clientes como Claude Desktop/Code; el agente es usable también por CLI y notebook.

---

3. Memoria y recuperación de contexto (IE3, IE4)

Memoria de corto plazo (src/memoria/corto_plazo.py): el checkpointer de LangGraph persiste el estado del grafo tras cada nodo, agrupado por thread_id. En conversaciones multi-turno el historial de veredictos y decisiones previas permanece disponible para los turnos siguientes; con la opción MEMORIA_CHECKPOINT (SqliteSaver) las tareas se reanudan incluso entre sesiones (flujos prolongados).

Memoria de largo plazo (src/memoria/largo_plazo.py): cada auditoría completada se guarda como experiencia {publicación, categoría, veredicto} en la colección Chroma experiencias_auditoria (Chroma, 2025). En casos nuevos, buscar_experiencias() recupera por similitud vectorial las experiencias más parecidas y las inyecta en los prompts del redactor. El escenario 4 de la demo (data/eval/ev2/evidencia_memoria_largo_plazo.json) confirma que una corrida posterior recupera experiencias de ejecuciones anteriores (n ≥ 1).

Recuperación semántica de contexto (IE4): reutilizada de la Evaluación 1 sin modificaciones: 312 chunks (Ley 21.442 completa + 5 RIC SEC + 1 Oficio Circular), embeddings multilingües Sentence-BERT (Reimers y Gurevych, 2019), persistencia Chroma y fallback automático del filtro por categoría (bug conocido de Chroma 1.5.x limita $contains con strings). La calidad del retrieval se evaluó con LLM-as-judge en la EV1 (v2 > v1: +20 % de relevancia, +46 % de completitud; ver docs/judge-results.md).

---

4. Planificación y toma de decisiones (IE5, IE7)

El planificador (supervisor) clasifica la solicitud en auditoria o consulta_normativa y emite un plan JSON priorizado, donde cada paso lleva prioridad y justificación. Ejemplo real para la solicitud "Necesito arreglar el portón eléctrico del edificio":

```
{
  "tipo_tarea": "auditoria",
  "pasos": [
    {"nombre": "memoria", "prioridad": 1,
```

```
    "justificacion": "Recuperar experiencias previas de instalaciones de
                      portones electricos en edificios similares."},
   {"nombre": "auditoria", "prioridad": 2,
    "justificacion": "Evaluar el cumplimiento de la normativa electrica
                      y de seguridad aplicable a la instalacion."},
   {"nombre": "redaccion", "prioridad": 3,
    "justificacion": "Redactar el informe con observaciones,
                      recomendaciones y plan de accion."},
   {"nombre": "guardar_memoria", "prioridad": 4,
    "justificacion": "Persistir la experiencia para futuros casos."}
  ]
}
```

La priorización va de lo más barato/informativo (recuperar memoria) a lo costoso (auditoría con RAG + LLM). Un respaldo heurístico determinístico garantiza el plan ante fallas del LLM.

Decisiones adaptativas demostradas (bitácoras reales en data/eval/ev2/, 5/5 escenarios OK):

- Publicación vaga → el redactor cambia a modo proponer_revision en lugar de emitir el informe de incumplimiento.
- Pregunta normativa → ruta alterna sin auditoría (consulta_normativa) y respuesta redactada con citas.
- LLM del planificador indisponible → degradación a heurística en 21 ms, el flujo continúa íntegro.
- Error transitorio de una herramienta → el enrutador reintenta una vez y, si persiste, salta el paso y lo reporta el finalizador.

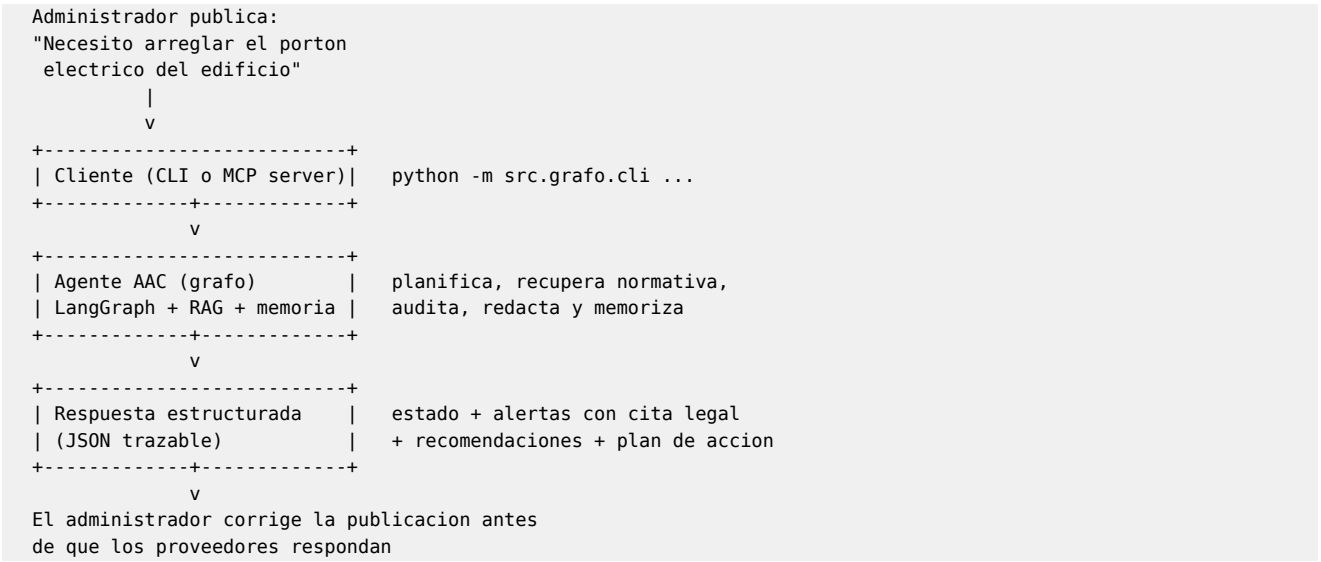
Ejemplo de flujo de trabajo (caso: portón eléctrico)

1. Entrada: "Necesito arreglar el portón eléctrico del edificio", categoría electricidad. 2. Planificación: el planificador clasifica auditoria y emite el plan priorizado de 4 pasos (arriba). 3. Memoria: se recuperan 3 experiencias previas de casos similares. 4. Auditoría: auditar() recupera los chunks relevantes (Ley 21.442 + RIC SEC) y devuelve alerta con 2 alertas citadas. 5. Redacción: como la publicación es concreta, se elige el modo informe_auditoria con observaciones y plan de acción. 6. Persistencia: la experiencia queda en experiencias_auditoria para casos futuros. 7. Salida: respuesta estructurada con estado, alertas con cita a la fuente legal y recomendaciones. Duración medida: 18,5 s.

---

5. Documentación técnica y flujos (IE6, IE8, IE9)

El flujo de trabajo organizacional que resuelve el agente:



- README ejecutable en el repositorio: instalación, indexado y ejecución (CLI agente, demo y MCP server) para un evaluador externo (IE8).
- Diagramas de orquestación en docs/diagrama-orquestacion.md (Mermaid: grafo general, flujo organizacional y secuencia con memoria) (IE8).
- Justificación de diseño por componente y su alineación con el flujo en docs/decisiones-diseno-ev2.md (IE9).
- Trazabilidad: bitácora por nodo en el estado y log JSONL por ejecución (data/agente_ejecuciones.jsonl).

---

6. Pruebas de software (evidencia)

- 46 tests pytest en verde (22 de la EV1 + 24 nuevos de herramientas, memoria, grafo y parser JSON), todos mockeados para ejecutarse sin costo en CI (GitHub Actions).
- Demo de 5 escenarios (scripts/demo_agente_ev2.py) corridos contra la API real de Groq, con outputs en

data/eval/ev2/: auditoría normal, publicación vaga, consulta normativa, memoria de largo plazo y degradación del planificador (5/5 OK).
- Smoke test HTTP del MCP server (scripts/test_mcp_endpoint.py): handshake, tools/list y tools/call funcionando.

7. Limitaciones y trabajo futuro

El corpus cubre Ley 21.442 y RIC SEC de uso más frecuente; ampliarlo hacia normas de gas/agua implica reindexar, no reescribir. El supervisor usa llamadas LLM secuenciales (3 por auditoría típica); la extracción de citas podría migrar a un modelo más pequeño de menor latencia. Como trabajo futuro: subagente revisar_cotizacion (reutiliza el patrón de auditoría), deploy del MCP server y dashboard de evaluaciones históricas.

8. Declaración de uso de IA (obligatorio)

Se usaron herramientas de IA generativa como apoyo: generación y refactorización de código boilerplate, revisión de redacción de documentación técnica y creación de diagramas en formato Mermaid. Todas las decisiones de diseño, análisis de resultados y justificaciones fueron revisadas y validadas en este trabajo; el comportamiento del agente se verificó con corridas reales y pruebas automatizadas (evidencia en el repositorio). Citado conforme a https://bibliotecas.duoc.cl/ia.

9. Reflexión individual (sin apoyo de IA — obligatorio)

[NOMBRE DEL ESTUDIANTE]: [Escribir aquí tu reflexión personal: qué aprendiste, cuál fue tu contribución concreta, qué dificultades enfrentaste y cómo las resolviste. Redactar SIN apoyo de IA según el reglamento de la evaluación.]

10. Referencias (APA 7)

- Chroma. (2025). Chroma documentation. https://docs.trychroma.com/
- Groq. (2025). Groq API documentation. https://console.groq.com/docs
- LangChain AI. (2025). LangGraph documentation. https://langchain-ai.github.io/langgraph/
- Ministerio de Vivienda y Urbanismo. (2021). Ley N° 21.442 sobre copropiedad inmobiliaria. Diario Oficial de la República de Chile. https://www.bcn.cl/leychile
- Reimers, N., y Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Proceedings of EMNLP. https://www.sbert.net/
- Superintendencia de Electricidad y Combustibles. (s. f.). Reglamento de Instalaciones de Consumo y Pliegos Técnicos (RIC 1, 2, 3, 5 y 6). https://www.sec.cl
- Duoc UC. (s. f.). Bibliotecas: uso de inteligencia artificial. https://bibliotecas.duoc.cl/ia